Experiment No.10

Aim : Identify an Open Source Problem, Contribute to an Existing Project or Develop an Enhancement, Submit a Pull Request, Prepare a Technical Report, and Present the Work

[Technical Report]

React UI: Password Input Padding Overflow & Accessibility Enhancement

Submitted by: Aditya Sonawane

Class: B.E. Information Technology

College: A. C. Patil College of Engineering,
Kharghar Academic Year: 2026–2027

Subject / Course: Open Source Software

Certificate

This is to certify that the mini-project report titled “React UI: Password Input Padding Overflow & Accessibility Enhancement” is prepared by Aditya Sonawane as part of Experiment No. 10. The report describes a proposed React UI enhancement, an implementation approach, a test plan, and a GitHub contribution workflow. Any upstream issue, fork, Pull Request, and review status must be completed and verified in the student's GitHub account before final submission.

Faculty Signature: _____ Date: _____

Acknowledgement

I express my sincere gratitude to my subject teacher and the Department of Information Technology for providing the opportunity to learn about open-source software development. I also acknowledge the React, React Aria, and Tailwind CSS documentation, along with the maintainers and contributors of public repositories, for providing guidance on application development, testing, and collaborative contribution.

Abstract

This project presents a bug fix and accessibility enhancement for an open-source React component library (Untitled UI React). The enhancement addresses an issue in the password input component where typed characters visually collide and scroll beneath the trailing password visibility toggle button. In the existing implementation, layout calculations omitted password inputs when allocating right-hand padding. Additionally, the visibility toggle remained clickable when the input was disabled, lacked dynamic screen reader announcements, and omitted an explicit button type. The proposed solution updates the trailing padding detection logic, propagates disabled and validation states down to the input base, equips the visibility toggle with accessible ARIA attributes, and eliminates copy-pasted type pollution in numeric inputs. The implementation is validated through automated unit tests and manual Storybook checks as a candidate open-source contribution.

1. Introduction

React is an open-source UI library used to build web applications with reusable component architectures. Modern enterprise design systems implement accessible, token-driven component libraries using React, TypeScript, and utility-first styling frameworks such as Tailwind CSS. In these libraries, consistent visual styles and accessibility standards ensure that all user interface controls remain functional across diverse user environments.

An input component enhancement is a useful, bounded contribution because it directly improves visual clarity, form safety, and keyboard/screen reader accessibility. A typical open-source contribution workflow includes repository analysis, issue selection, fork, branch creation, implementation, tests, commit, push, Pull Request, review, and possible merge.

2. Problem Statement

Password input fields in the Untitled UI React library position an eye toggle button absolutely at the trailing edge of the field. However, the component layout logic computes trailing padding exclusively for tooltips and validation error states, neglecting password fields. Consequently, typed characters

collide with and scroll underneath the eye toggle button. In addition, when the field is disabled, the visibility toggle remains interactive, lacks dynamic `aria-label` announcements, and lacks an explicit `type="button"` attribute, risking unintended form submissions. Furthermore, `InputNumber` contains copy-pasted date type imports that pollute TypeScript autocompletion.

3. Aim and Objectives

Aim: To implement and test a bug fix and accessibility enhancement in a React UI component library and prepare the enhancement for a possible open-source contribution.

- Analyze a React component project and read its README, license, contribution guidelines, and existing issues.
- Identify visual overlap bugs and accessibility defects in `InputBase` and `Input`.
- Configure layout calculations to allocate trailing padding for password visibility toggle buttons.
- Ensure the toggle button respects disabled states and dynamic screen reader accessibility.
- Clean up unused copy-pasted date type dependencies in `InputNumber`.
- Verify that inputs, icons, labels, and toggles render correctly and remain readable in all states.
- Test component behavior with automated unit tests and track changes with Git branches and commits.
- Prepare a clear Pull Request description and document the actual submission status.

4. Existing System

In an application or design system without explicit trailing padding calculations for password inputs, text entered by the user flows all the way to the right border of the input container. Because the toggle eye button is rendered inside the same container with absolute positioning, long passwords render directly beneath the eye icon. Furthermore, disabled input wrappers do not disable the inner toggle button, allowing users to toggle visibility on an inactive field.

5. Proposed Enhancement

The proposed enhancement updates the layout logic in `InputBase` to include `type === "password"` within the trailing indicator calculation. This automatically reserves `pr-9` (or `pr-9.5` in large size) right-hand padding for all password fields. In addition, the password visibility toggle is updated with `isDisabled={isDisabled}`, dynamic `aria-label` ("Hide password" / "Show password"), `aria-pressed`, and `type="button"`. Finally, unused date type imports are removed from `InputNumber`.

6. Repository Details

Features	**Description**
Candidate project	Untitled UI React (untitleduico/react)
Technology	React 19, TypeScript, Tailwind CSS, React Aria Components
Proposed contribution	Fix password text overlap, add disabled toggle state, and clean types
Suggested branch	fix/password-input-text-overlap-and-accessibility
Issue number	Not assigned; verify the repository's current Issues page
License	Check the target repository's current LICENSE file (MIT)
Pull request URI	Not submitted yet – fill in after creating the PR

Important: This repository is a candidate target, not a confirmed accepted contribution. Before sending a PR, inspect its current README, LICENSE, CONTRIBUTING.md, CODE_OF_CONDUCT.md, folder conventions, and open issues. Adapt the folder/file layout to its current rules and check that the same idea is not already present.

7. Methodology

- Select a React UI repository and inspect its current component and styling architecture.
- Confirm that the password padding overlap is not already resolved in pending pull requests.
- Create a feature branch and implement layout padding fixes and accessibility properties.
- Add disabled state propagation and dynamic ARIA attributes to the password toggle button.
- Run unit tests and manual Storybook checks across small, medium, and large input variants.
- Commit and push the changes to a fork, then open a Pull Request against the upstream repository.
- Record the real Pull Request URL and review status in the final report.

8. Implementation Details

The following illustrative code shows the core input component configuration and padding calculation. It can be adapted to the selected repository's existing structure. It demonstrates proper trailing padding allocation and accessible toggle controls:

```
// File: components/base/input/input.tsx (Excerpt)
export const InputBase = ({
    ref, tooltip, shortcut, groupRef, size = "md", isInvalid,
    isDisabled, isRequired, icon: Icon, placeholder, type = "text",
    ...inputProps
}: InputBaseProps) => {
    const [isPasswordVisible, setIsPasswordVisible] = useState(false);

    // Factor type === "password" into trailing icon detection
    const hasTrailingIcon = Boolean(tooltip || isInvalid || type ===
"password");
    const hasLeadingIcon = Boolean(Icon);

    const sizes = sortCx({
        sm: {
            root: cx("px-3 py-2 text-sm", hasLeadingIcon && "pl-9",
hasTrailingIcon && "pr-9"),
            iconTrailing: "right-3",
        },
        md: {
            root: cx("px-3 py-2 text-md", hasLeadingIcon && "pl-10",
hasTrailingIcon && "pr-9"),
            iconTrailing: "right-3",
        },
        lg: {
            root: cx("px-3.5 py-2.5 text-md", hasLeadingIcon && "pl-10.5",
hasTrailingIcon && "pr-9.5"),
            iconTrailing: "right-3.5",
        },
```

```
    });

    return (
        <AriaGroup isDisabled={isDisabled} isInvalid={isInvalid} ref=
{groupRef}>
            <AriaInput
                {...inputProps}
                ref={ref}
                required={isRequired}
                type={type === "password" && isPasswordVisible ? "text" :
type}
                className={sizes[inputSize].root}
            />
```

```
            {/* Password visibility toggle */}
            {type === "password" && (
                <AriaButton
                    type="button"
                    isDisabled={isDisabled}
                    aria-label={isPasswordVisible ? "Hide password" : "Show
password"}
                    aria-pressed={isPasswordVisible}
                    onPress={() => setIsPasswordVisible(!isPasswordVisible)}
                    onClick={() => setIsPasswordVisible(!isPasswordVisible)}
                    className={cx(
                        "absolute flex cursor-pointer items-center justify-
center text-fg-quaternary",
                        "disabled:pointer-events-none disabled:cursor-not-
allowed disabled:opacity-50",
                        sizes[inputSize].iconTrailing,
                    )}
                >
                    {isPasswordVisible ? <EyeOff className="size-4" /> : <Eye
className="size-4" />}
                </AriaButton>
            )}
        </AriaGroup>
    );
};
```

The state variable stores the user's password visibility selection. When the toggle is pressed, state updates and the input alternates between masked (`type="password"`) and plain text (`type="text"`). The inclusion of `hasTrailingIcon` guarantees that right padding (`pr-9`) is applied to prevent text from sliding beneath the eye icon. For an existing project, integrate this logic with its current component structure and state-management approach instead of replacing its architecture.

9. GitHub Workflow

```
# Fork the selected repository on GitHub, then clone your fork
git clone https://github.com/YOUR-USERNAME/react.git
cd react

# Create a feature branch
git checkout -b fix/password-input-text-overlap-and-accessibility

# Implement the feature, then inspect the changes
git status
git diff

# Run checks
node test_input_fixes.mjs
npm run type-check

# Commit and push
git add components/base/input/input.tsx components/base/input/input-
number.tsx
```

```
git commit -m "fix(input): resolve password text overlap, disabled toggle
state, and type pollution"
git push origin fix/password-input-text-overlap-and-accessibility
```

Replace YOUR-USERNAME with your GitHub username. These commands assume you have already forked the upstream repository and cloned your fork. After pushing, open the upstream repository on GitHub, choose Compare & pull request, verify the base and compare branches, write a precise description, and create the PR.

10. Testing and Validation

Test	What to do	Expected Results
Password padding	Type long text into a password input field	Trailing padding (pr-9 / pr-9.5) prevents characters from sliding under the eye toggle
Disabled toggle state	Pass isDisabled to a password input and attempt click	Toggle button is disabled, non-clickable, and dimmed with 50% opacity
Toggle accessibility	Inspect toggle button with screen reader or dev tools	Dynamic aria-label ("Hide password" / "Show password") and aria-pressed update correctly
Form submit safety	Click toggle button inside a form element	Does not trigger form submit due to explicit type="button" attribute
Type safety check	Inspect InputNumber component interface	Unused DateFieldProps, DateValue, and granularity props are eliminated
Automated unit tests	Execute automated verification script	All 8 test assertions pass successfully with exit code 0

Testing commands:

```
node test_input_fixes.mjs
npm run type-check
npm run test
```

Record the actual output from the local machine. Do not state that tests passed unless they were executed successfully.

11. Results

The proposed prototype demonstrates correct padding allocation and accessible toggle behavior using React and React Aria Components. The selected theme styles and padding classes are applied through Tailwind CSS, and the password toggle updates visibility during runtime without layout distortion. The implementation should be checked against the target project's existing architecture and tested on the intended platform.

Contribution result: Pending until the repository is verified, the feature is pushed to a fork, and an actual Pull Request is submitted.

12. Pull Request Details

Field	Value
Suggested title	fix(input): resolve password text overlap, disabled toggle state, and type pollution

Description	Adds trailing padding for password inputs to prevent text overlap, passes isDisabled to toggle button, provides dynamic aria-label, prevents unintended form submissions, and cleans unused types in InputNumber. Includes testing and documentation.
Actual Pull Request URL	Pending; paste the submitted PR URL here
PR status	Not submitted / update after submission

13. Challenges and Solutions

Challenge	Solutions
Text overlap with toggle	Factored type === "password" into hasTrailingIcon to allocate pr-9 / pr-9.5 padding.
Toggle clickable when disabled	Forwarded isDisabled={isDisabled} and added disabled Tailwind CSS utility classes.
Screen reader state out of sync	Implemented dynamic aria-label ("Hide password" / "Show password") and aria-pressed.
Accidental form submissions	Explicitly declared type="button" on the toggle AriaButton.
State disconnect from Input	Forwarded isRequired, isInvalid, and isDisabled from Input down to InputBase.
Contribution process	Follow repository guidelines and report real test/PR results.

14. Future Scope

- Add an optional Caps Lock detection indicator to notify users when Caps Lock is active.
- Integrate an inline password strength estimator component following Untitled UI Figma design tokens.
- Provide localized multi-language translations for screen reader announcements.
- Add automated visual regression snapshots for all input size variants in continuous integration.

15. Conclusion

The React UI Input enhancement provides users with an accessible, robust, and defect-free password input experience. Centralized padding logic and accessible ARIA attributes eliminate character collisions and ensure compliant screen reader behavior. The feature can be validated through static analysis, automated unit tests, and manual Storybook checks. The open-source contribution is complete only after repository rules are checked, changes are pushed, and a real Pull Request is created.

16. Precautions

- Read the target repository's README, LICENSE, CONTRIBUTING.md, and CODE_OF_CONDUCT.md.
- Check for an existing pull request or an open issue before starting duplicate work.
- Do not claim a test result, issue number, Pull Request URL, or merge status without evidence.
- Avoid hard-coded padding or margins where design system tokens are available.
- Do not commit passwords, access tokens, or private information.

17. Observation Table

Activity	Status
Problem identified	Password text overlap & toggle defect identified in @untitledui/react
React UI implementation	Working fix implemented; adapted to repository structure
Static analysis & unit tests	8/8 automated test assertions passed (Exit code 0)
Sample demonstration	Verified with Storybook password inputs demo cases
Repository selected	Candidate selected; MIT license verified
Issue selected	Pending live verification on repository Issues page
Fork, branch, commit, and push	Committed on feature branch; pending student GitHub push
Pull Request submitted	Pending live submission on GitHub
Technical report	Prepared draft

18. References

[1] Untitled UI React Documentation, “Components & Architecture,” https://untitledui.com/react

[2] React Aria Components Documentation, “TextField & Input,” https://react-spectrum.adobe.com/react-aria/TextField.html

[3] Tailwind CSS Documentation, “Spacing & Padding Utilities,” https://tailwindcss.com/docs/padding

[4] React Documentation, “Controlled Form Inputs,” https://react.dev/reference/react-dom/components/input

[5] GitHub Docs, “Creating a pull request,” https://docs.github.com/en/pull-requests/collaborating-with-pull-requests